Admission Control for LLM Inference Serving: A Systematic Review

ANONYMOUS AUTHOR(S)

Serving large language models at scale is a distributed systems problem whose overload behavior is governed at the ingress. We conduct an adapted-PRISMA systematic review of fifty-five admission and admission-adjacent works published between 2022 and 2026 across OSDI, SOSP, NSDI, ASPLOS, EuroSys, MLSys, SoCC, ICML, NeurIPS, HPCA and arXiv, and report three findings. First, admission sits at a different layer from the one the serving literature studies: of nine LLM serving papers audited across a seven-venue slice, all place the overload response inside the engine, through batching, migration, placement, autoscaling or memory management, and none takes a decision at the ingress. Second, of sixteen admission-primary papers, no two share a baseline set: policies are measured against the absence of admission control and almost never against one another. Third, placing all sixteen on our axes succeeds on six of seven properties and fails on one. Unit, setting, objective, the quantity a policy watches, the metadata it assumes, and whether it is deployable in front of an unmodified engine are all recoverable from the published text; the observed range of the quantity the policy reads is reported by none of them. Selection therefore fails on a single missing fact, and it is the fact that determines whether a reported improvement is interpretable at all. We contribute a taxonomy of unit, setting, SLO awareness, signal quantity and portability, with a master classification carrying GRADE-style evidence ratings; formal definitions of the admission-decision, goodput-under-admission and fairness-index primitives; seven reporting items whose inconsistency blocks cross-paper comparison; a mapping of eight public production incidents to admission failure modes; and seven open problems, each grounded in published peer review or in a primary source's own limitations.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → *Software performance*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: Admission control, LLM inference serving, overload control, service-level objectives, systematic review, goodput, queueing

ACM Reference Format:
Anonymous Author(s). 2026. Admission Control for LLM Inference Serving: A Systematic Review. *ACM Comput. Surv.* 0, 0, Article 0 (September 2026), 26 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

On November 8, 2023, OpenAI's inference fleet crossed a soak point after a day of unusually heavy completion traffic. Routing nodes reached memory limits and failed readiness checks. Additional replicas were added, then rolled back. Recovery required cutting inbound traffic, redeploying, and slowly ramping load back up. The postmortem named graceful load shedding through stronger rate-limit controls as the required follow-up. Two years later, on January 21, 2026, Anthropic's Sonnet 4.5 tier degraded first in latency and then in error rate over roughly ninety minutes. The postmortem cited insufficient compute capacity for demand and named earlier detection of capacity constraints as a required change. Cloudflare [10, 11], Azure, and other production operators have documented similar sequences.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
0360-0300/2026/9-ART0 $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Admission control is the decision made before a request enters the running batch. It has three actions: admit into service, hold in queue, or reject with a well-defined error contract. Figure 1 depicts the model. When admission is wrong, downstream batching, KV cache management, and preemption cannot recover lost capacity. Requests admitted past a soak point occupy memory until they miss their deadline, at which point they consume more capacity being cleaned up.

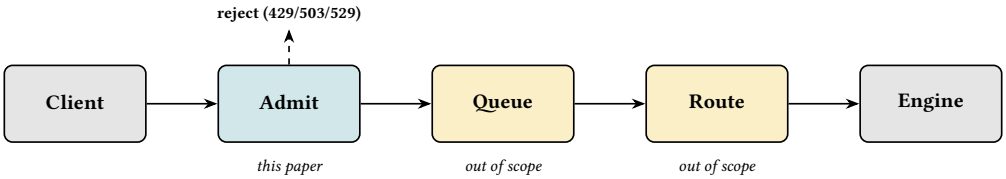


Fig. 1. The Admit–Queue–Route model. Admission is the decision that gates a request into the serving pipeline. Queue and Route are downstream boxes covered by prior surveys. This paper is about the first box.

The literature on LLM inference has produced systems papers on batching [2, 57], disaggregation [44, 60], memory management [29, 47], and scheduling [20, 50]. Existing surveys cover efficient serving broadly [19, 31, 59] or the queueing-theoretic view of LLM systems specifically [38]. Table 1 differentiates this survey from those prior works.

Contributions.

(1) **Systematic coverage** of fifty-five admission and admission-adjacent works over 2022–2026 following an adapted-PRISMA protocol (Section 4, Figure 4).
(2) **Unified taxonomy** with five axes and five sub-branches, plus short codes (UNIT-REQ/AGT/TEN, SIG-SCA/DG/LP/WCRT) used across the master table (Section 3).
(3) **Foundational definitions** of the admission decision, goodput-under-admission, and fairness-index primitives (Section 2.4).
(4) **Master classification** of all admission-primary and admission-adjacent works with GRADE-adapted evidence ratings (Section 6, Table 3).
(5) **Admission-signal overhead synthesis** that identifies the cost-versus-lift tradeoff across scalar, dual-gate, LP, and formal-WCRT signals (Section 7).
(6) **Reproducibility audit** finding seven of the sixteen admission-primary papers have released code, together with seven reporting items whose inconsistency blocks cross-paper comparison (Section 8).
(7) **Production evidence mapping** that ties eight public postmortems to admission failure modes (Section 9).
(8) **Structured research agenda** of seven open problems, each grounded in either public peer review or the primary source's own limitations (Section 10).

Headline observations. First, of nine scheduling papers we audited across OSDI, SOSP, NSDI, ASPLOS, EuroSys, MLSys, and HPCA between 2022 and 2026, all absorb overload inside the engine, through batching, migration, placement, autoscaling or memory management, and none takes a decision at the ingress. This reflects the scope each paper sets for itself rather than an omission. Token Pools (CAIS 2026) sits outside this seven-venue slice and does gate at the API gateway, which suggests ingress admission has been treated as an infrastructure concern rather than an engine-level research question. Second, of sixteen admission-primary papers we audited, no two share a baseline set, and the only baselines shared by more than one paper are unconfigured serving

engines and classical scheduling disciplines. Exactly one published admission system, SHEPHERD [58], is used as a baseline by two papers (QLM and Ascendra). Third, the taxonomy accommodates every corpus work that proposes or embeds an admission mechanism, twenty-eight in total (Table 3), including the five added during verification (Token Pools, ProServe [22], SAGA, Scorpio and FairServe), with four sub-branches added during our reading, none of which required backing out an original axis.

Roadmap. Section 2 covers the inference request lifecycle, the Admit-Queue-Route model, and the foundational definitions (§2.4). Section 3 introduces the five-axis taxonomy with short unit and signal codes. Section 4 describes the systematic-review method with a PRISMA flow diagram. Section 5 synthesizes each policy cluster. Section 6 presents the master classification with GRADE-adapted evidence ratings. Section 7 presents the admission-signal overhead cross-paper synthesis. Section 8 audits reproducibility and identifies the seven divergent reporting items. Section 9 maps production incidents to admission failure modes. Section 10 lists the seven open problems. Section 11 concludes.

1.1 Related surveys and differentiation

Four broad surveys of LLM inference serving predate this work. Table 1 summarizes each and states what this paper adds.

Table 1. Differentiation from prior surveys.

Prior survey	Scope	Admission depth	Taxonomy axes	Grounded open problems
Zhen et al. [59] INLG 2025	Efficient serving (batching, KV, sched., fair.)	One subsection	Task-oriented	No
Heisler et al. [19] TechRxiv 2025	Scheduling techniques & trade-offs	Component	Technique-oriented	No
Li et al. [31] IEEE HPEC 2024	Recent serving advances	In passing	Chronological	No
Mitzenmacher & Shahout [38] SIGMETRICS 2025	Queueing theory position paper	Tutorial	Queueing-theoretic	Enumerates 25
Ma et al. [36] SSRN 2026	Load balancing as a coupled control problem	Routing, not ingress	Control-theoretic	No
This paper	**Admission as primary axis**	**Full**	**4 axes + 4 sub-branches**	**7 grounded**

Zhen et al. and Heisler et al. treat admission as one component among many. Li et al. is a survey of recent advances organized chronologically. Mitzenmacher and Shahout is a position paper that enumerates open queueing problems without a taxonomy or per-paper review. None conducts a reproducibility audit or organizes the literature by admission granularity, SLO awareness, signal structure, or concurrency context.

2 BACKGROUND

2.1 The inference request lifecycle

A large language model inference request passes through prefill, decode, and completion. Prefill computes the key-value tensors for every input token. Decode generates output tokens one at a

time, reading the accumulated KV cache and appending a new pair per step. Continuous batching [57] reuses a single GPU-resident engine across many concurrent requests by interleaving iterations at token granularity. PagedAttention [29] manages the KV cache in fixed-size blocks to reduce fragmentation and enable prefix sharing.

The KV cache is the memory hierarchy's dominant consumer. It grows monotonically with generated tokens during a request's decode, and it persists across turns for multi-turn agents. When aggregate KV footprint approaches available GPU memory, the engine must either evict (recompute later), swap (page to CPU or disk), or preempt (kill the request and return an error).

2.2 The Admit–Queue–Route model

Every request that reaches an inference cluster passes through three decision boxes before it produces its first token (Figure 1). **Admit** decides whether to accept the request into service. **Queue** decides in what order accepted requests are served. **Route** decides which replica or instance receives the request. This paper is about the first box.

The error contract matters. HTTP 429 signals a client-side rate limit exceeded. HTTP 503 signals a temporarily unavailable service. Anthropic uses HTTP 529 to signal a provider-overloaded state distinct from either [3]. A well-designed admission policy chooses among these codes based on which signal the client's backoff logic should trust.

2.3 Service-level objectives

The community reports admission behavior against a small set of metrics. Time-to-first-token (TTFT) is the wall-clock delay between arrival and the first generated token. Time-between-tokens (TBT), sometimes called inter-token latency or time-per-output-token (TPOT), is the delay between consecutive generated tokens. Goodput is the completed-request rate that met their service-level objective, distinct from raw throughput [60]. For agentic workloads, agent-completion rate measures the fraction of multi-turn sessions that finish within an end-to-end budget.

2.4 Foundational definitions

We fix three primitives that recur across the surveyed literature. Each formalizes a concept that admission-primary papers rely on but name inconsistently, and each is referenced explicitly in the master classification (Table 3) and the admission-signal overhead synthesis (Section 7).

Definition 1 (Admission Decision)**.** Given an arriving request r with arrival time t_r, feature vector $x_r \in \mathcal{X}$, and a system state $s_t \in \mathcal{S}$, an *admission policy* is a mapping $\pi : \mathcal{X} \times \mathcal{S} \to \{\text{ADMIT}, \text{DEFER}(\tau), \text{REJECT}(c)\}$ where $\tau > 0$ is a retry delay and $c \in \{429, 503, 529\}$ is an HTTP error code whose semantics distinguish client-throttled, service-unavailable, and provider-overloaded outcomes.

Definition 2 (Goodput-under-admission)**.** Let $\mathcal{R}$ be the set of requests offered to the system in interval $[0, T]$, $\mathcal{A} \subseteq \mathcal{R}$ the admitted subset, and $\text{SLO}(r)$ a binary indicator of whether request r completed within its service-level objective. Goodput-under-admission is

$$G_\pi = \frac{|\{r \in \mathcal{A} : \text{SLO}(r) = 1\}|}{|\mathcal{R}|} \tag{1}$$

This is the fraction of the *offered* load that both survived admission and met its SLO. It differs from throughput (which ignores SLO attainment) and from SLO attainment (which conditions on admitted requests only and hides work shed by an aggressive policy).

The predicate $\text{SLO}(r)$ is parametrized by an SLO tuple that depends on workload class. For interactive request-level workloads it is typically the conjunction ($\text{TTFT}_r \le \tau_{\text{TTFT}} \wedge \max_t \text{TBT}_{r,t} \le$

τ_{TBT}). For agent-session workloads it is a job-level end-to-end budget ($\text{JCT}_r \le \tau_{\text{JCT}}$), optionally with per-stage sub-budgets as in SLOs-Serve. Cross-paper comparison of G_π requires the SLO tuple to be specified, since aggregating across different tuples produces incomparable numbers; item 7 of Section 8.1 asks for the SLO tuple to be stated per workload class.

Definition 3 (Fairness Index)**.** For n tenants (or agents, or classes) with weights w_i and served tokens s_i over interval $[0, T]$, we use *Jain's fairness index* [24] in weighted form. Jain, Chiu, and Hawe introduced this index in 1984 as a scale-independent measure of how equitably a set of quantities is distributed across users; it has been the standard fairness primitive in network scheduling and computer-systems resource allocation for four decades. The weighted form is:

$$J(\mathbf{s}, \mathbf{w}) \;=\; \frac{\left(\sum_{i=1}^{n} s_i/w_i\right)^2}{n \sum_{i=1}^{n} (s_i/w_i)^2} \tag{2}$$

with $J \in [1/n, 1]$; $J = 1$ denotes perfect proportional fairness and $J = 1/n$ denotes maximal concentration on a single tenant. Chosen as the primitive because it composes cleanly across the three fairness types in the taxonomy (task, tenant, priority-weighted).

The Jain index is a coarse instrument. Equinox [34] and related utility-centric work argue that equal tokens do not imply equal user experience, and that a fairness measure grounded in predicted response quality or user-perceived latency is more practitioner-relevant. Utility-centric measures resist cross-paper composition because each defines its own utility function, so we retain the Jain index as the comparability primitive and note this substitution risk explicitly rather than hide it.

2.5 Why admission control is hard

Ao, Dong, Luo, and Simchi-Levi [6] formalize the core problem. In a discrete-time model of continuous batching under a memory budget, the service process itself creates future capacity pressure because each active request's KV footprint grows with each generated token. They prove that the natural eviction-free equilibrium is unstable under any admission rule that only checks instantaneous memory feasibility. Along the resulting limit cycle the throughput ratio equals

$$\rho^* \;=\; \frac{2\ell_0 + \ell_1 + 1}{2(\ell_0 + \ell_1)} \tag{3}$$

which lies in $(1/2, 1)$ and converges to $1/2$ as $\ell_1/\ell_0 \to \infty$. Their Theorem 3 further shows that stability depends on the greatest common divisor of active decoding lengths: if $\gcd(\ell_{1,1}, \ldots, \ell_{1,K}) = g > 1$, unstable modes appear on g-th roots of unity and synchronize into eviction cycles; if $g = 1$, all roots lie strictly inside the unit disc and the eviction-free equilibrium is globally asymptotically stable. This is a workload-arithmetic property of admission decisions that no production system currently exposes as a signal.

3 A TAXONOMY OF ADMISSION-CONTROL POLICIES

We organize admission-control policies along four *descriptive* axes (unit of admission, setting, SLO awareness, signal used) and one *classificatory* dimension (decision moment) used only in the master classification table (Section 6). The descriptive axes carry the substantive taxonomy claims; the classificatory dimension is a fold that captures where in the serving pipeline the admission decision is taken. We call this a five-axis taxonomy throughout and reserve the term "three-level classification" for the master table's compact form.

Decision moment (folded). Every admission decision can be located at one of three points in the serving pipeline: under ING (*ingress*), the decision is taken before the request enters any server-side queue, based on request features and system state at arrival time; under QB (*queue boundary*),

it is taken when the request is popped from a waiting queue for batch admission, informed by both the request and the current batch state; under MF (*mid-flight*), it is taken during the request's decode, either as a preemption to a lower priority class or as a reclamation of committed KV cache. The master table uses these three codes because they discriminate sharply across the corpus; the descriptive Setting axis (online/offline/hybrid) collapses under this coding because the majority of surveyed systems operate at ING or QB regardless of setting.

Figure 2 places the three moments on the serving pipeline. The coding is not only descriptive: the moment at which a policy acts determines what state it can read, and in the systems surveyed here it also determines whether the policy sits in front of the engine or within it.

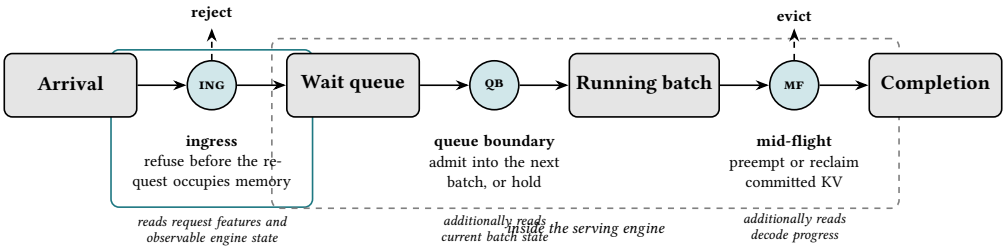


Fig. 2. Where an admission decision is taken. ING (ingress) acts before the request consumes engine memory, and reads only the request and whatever state the engine already exposes. QB (queue boundary) acts as the request is drawn into a batch and additionally reads batch state; MF (mid-flight) acts during decode and additionally reads the request's own progress. The later moments read more state, and in the systems surveyed here they are implemented inside the engine rather than in front of it.

Figure 3 shows the two most-cited descriptive axes (unit × SLO awareness) plotted against each other with representative policies placed on the grid.

3.1 Axis 1: Unit of admission

Admission decisions can be made at three granularities. **Request-level** (UNIT-REQ) policies admit or reject each incoming prompt independently (QLM [46]; Chronos [37]; FastServe [54]; SOLA [20]). **Agent-session-level** (UNIT-AGT) policies admit or reject entire multi-turn agent trajectories (CONCUR [8]; Continuum [32]; ThunderAgent [26]). **Tenant-level** (UNIT-TEN) policies gate admission per user or per organization (VTC [49]).

3.2 Axis 2: Setting

Online serving handles interactive latency-sensitive workloads (most of the field). Offline batch handles throughput-oriented inference where per-request deadlines are relaxed. CONCUR is explicitly offline per its ICML 2026 meta-review. Hybrid settings mix both.

Sub-branch: Concurrency context. We add a sub-branch after finding that Clairvoyant [51] solves an admission problem that server-side policies cannot generalize to. Concurrent-batch servers (vLLM, SGLang, TensorRT-LLM) admit at iteration granularity, contend for KV blocks across many active requests, and can preempt mid-decode. Serial edge backends (Ollama, llama.cpp) execute one request at a time. Head-of-line blocking is the dominant tail-latency pathology at the edge, and predictive shortest-job-first admission is the natural response.

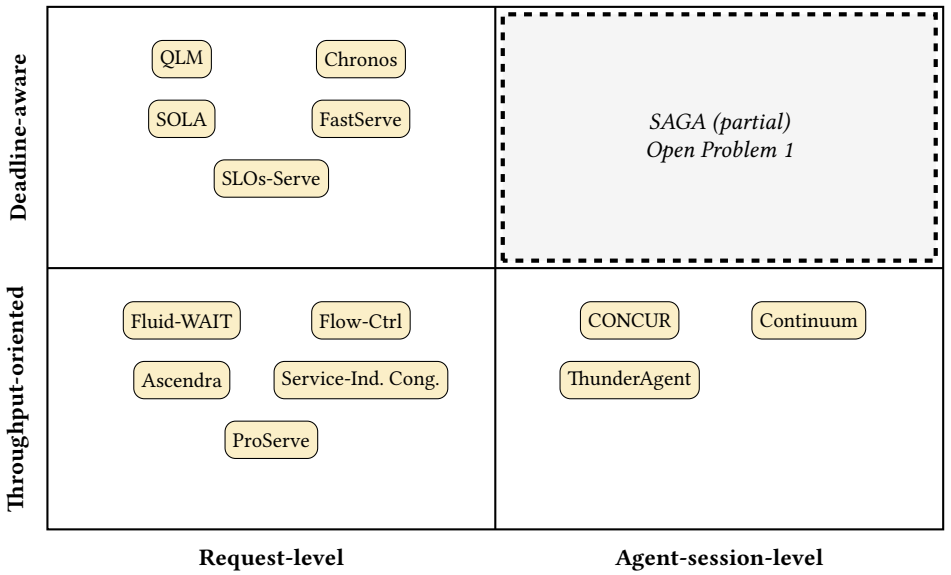


Fig. 3. Policy taxonomy on Unit of admission (horizontal) × SLO awareness (vertical). The upper-right quadrant (agent-session × deadline-aware) is partially occupied by SAGA (arXiv:2605.00528), which satisfies agent-level scheduling and per-task fairness but does not establish explicit TTFT/TBT deadlines, online serving mode, or per-tenant fairness. This is the seed for Open Problem 1.

3.3 Axis 3: SLO awareness

Deadline-aware policies condition admission on TTFT or TBT bounds (Chronos, SOLA, FastServe). Throughput-oriented policies maximize aggregate goodput without per-request deadlines (CONCUR, Continuum). Fairness-oriented policies enforce per-tenant or per-task equity (VTC, FairBatching, Equinox). Cost-oriented policies minimize dollars or energy per token.

Sub-branch: SLO granularity. SLOs-Serve [9] exposes a design axis nobody else covers. Job-level SLOs specify a single deadline per multi-turn agent job. Request-level SLOs specify one deadline per prompt (most of the field). Per-stage SLOs specify distinct deadlines per stage of a multi-turn application. Only SLOs-Serve models the per-stage case.

Sub-branch: Fairness type. Task-fairness allocates GPU cycles equitably between prefill and decode tasks within one batch (FairBatching [35]). Tenant-fairness equalizes served tokens or SLO attainment across users (VTC [49], Equinox [34]). These share no mechanism.

3.4 Axis 4: Signal quantity

Every admission policy compares some measured quantity against a threshold. Which quantity is the axis a practitioner can answer about their own deployment without having read any of the policies, because it asks what moves when their system is overloaded rather than how a given paper is built. Across the sixteen admission-primary works the quantities are:

KV pressure occupancy of the key-value cache, as a fraction of what the deployment has allocated (Fluid-WAIT, CONCUR, Continuum, AQUA).

Queue depth the number of requests waiting to enter a batch (QLM, FastServe, Fluid-WAIT, Continuum).

Deadline slack time remaining against a per-request target, compared with an estimate of time required (Chronos, SOLA, SLOs-Serve, Scorpio, ProServe).

Predicted length an estimate of how many tokens the request will generate, from a model over the prompt (FastServe, Scorpio, Clairvoyant).

Wait estimate a prediction of how long the request will queue, distinct from how many are queued (QLM).

Batch state the composition of the batch now forming, rather than a property of the arriving request (SOLA, FairBatching).

Arrival rate the offered load itself, with no engine-side quantity read (Flow-controlled scheduling).

Analytic no runtime signal: admission follows from a stability or reward argument (Service-Induced Congestion, Shen et al.).

The distribution matters for what the axis can do. Deadline slack and KV pressure dominate, but they are not interchangeable: a deadline-slack policy needs requests to arrive carrying deadlines, and a KV-pressure policy needs the cache to actually fill. Section 3.6 develops the second condition.

Sub-branch: Signal structure. Orthogonal to which quantity is read is how the policy combines readings into a decision. We identify four structural forms, each with a short code we use throughout the master classification (Table 3). The structure describes policy design rather than deployment fit, so it is a sub-branch rather than a primary axis: a reader cannot place their own system on it.

Scalar heuristics (SIG-SCA) compare a single quantity to a threshold (KV utilization for FluidWAIT; queue depth for QLM; predicted output length for Clairvoyant).

Dual-gate conjunctions (SIG-DG) require multiple signals to agree. CONCUR declares congestion only when both KV utilization is high and prefix cache hit rate is low, because high utilization with high hit rate is healthy reuse rather than thrashing:

$$\text{congested} \iff \big(U_t > U_{\text{high}}\big) \wedge \big(H_t < H_{\text{thresh}}\big) \tag{4}$$

with fixed hyperparameters $U_{\text{high}} = 0.5$, $H_{\text{thresh}} = 0.2$. Admission at agent granularity is adjusted via additive-increase-multiplicative-decrease (AIMD) with $\alpha = 2$, $\beta = 0.5$ per control interval.

Linear-program-based composite signals (SIG-LP) treat admission as one variable in a joint optimization. QLM formulates a linear program over virtual queues, minimizing total SLO-violation penalty:

$$\min \sum_{i \in \mathcal{R}} \max\big(0,\ W_{q,i} - \text{SLO}_i\big) \tag{5}$$

subject to one-to-one request-group-to-position and model-transition constraints; solvable in about 5 s at 400,000 queued requests using Gurobi.

Formal worst-case-response-time bounds (SIG-WCRT) derive an admission test from real-time-systems theory. Chronos casts prefill as a sporadic job with worst-case execution time proportional to input length, then derives the closed-form bound

$$R_i \ \le\ \frac{\rho_P \cdot D_{\text{TTFT}}}{1 - \rho_P} + p_i(\alpha B + \gamma), \qquad \rho_P < 1 \tag{6}$$

where ρ_P is prefill utilization, D_{TTFT} is the TTFT deadline, $p_i = \lceil \ell_i / B \rceil$ is the number of chunks, α is prefill kernel throughput, B is chunk size, and γ is fixed overhead. Algorithm 1 of [37] rejects on arrival if $\rho_P \ge 1$, if the bound exceeds the TTFT deadline, or if a TBT capacity check fails.

3.5 Axis 5: Portability

The four axes above describe what a policy decides. This one describes what it costs to run, and it is the axis a practitioner is likely to apply first, because it can eliminate candidates before any question of fit arises.

Class A policies decide from the arriving request and state the engine already exposes: cache occupancy, queue length, counts of running and waiting requests. Such a policy can sit in front of an unmodified engine, as a proxy or gateway, and is deployable by anyone who can route traffic.

Class B policies need to influence which requests enter a batch, or to act on requests already decoding. That cannot be done from outside: it requires changes inside the engine's scheduler, and in practice a maintained fork.

The split follows from the decision moment (§3, folded): ING is Class A, while QB and MF are Class B. Of the sixteen admission-primary works, ten are Class A and six are Class B. A reader who cannot maintain an engine fork is choosing among ten, not sixteen, and that is knowable before reading any of them.

No paper in the corpus states its own class. It is derivable from the mechanism each describes, which is why we treat it as an axis rather than as reported metadata: the information is present in the text but never surfaced as a deployment property.

3.6 What the Signal axis predicts

The Signal axis has a consequence for how published results should be read. A policy discriminates by comparing a measured quantity against a threshold, so it can discriminate only when that quantity varies over the operating range of the deployment. Where the signal is effectively constant, the comparison returns the same answer for every arrival, and the policy admits or refuses uniformly regardless of how its threshold is set. Its observable behaviour in that case matches an uncontrolled baseline.

The two cases are not distinguishable from a results table. A policy whose signal never approached its threshold and a policy that evaluated a varying signal and declined to reject will report the same admission counts and the same downstream latencies. Separating them requires the observed range of the signal over the evaluation. We did not find that range reported in any paper we examined (Section 8.1, item 3).

This also bounds what the present survey can establish. Which deployments hold which signals constant depends on the engine, the model, and the workload, and is an empirical question rather than a bibliographic one. We do not answer it here, and we are not aware of published measurements that do; it is stated as Open Problem 7.

3.7 Placing the corpus, and where it fails

We applied the five axes to all sixteen admission-primary works, recording for each what it admits, when, against which quantity, what it assumes a request carries, and whether it is deployable without an engine fork. The exercise is a test of the axes as much as of the corpus: an axis that cannot be filled from the published text is not one a reader could use.

Five of the five fill. Every paper states its unit and setting; every paper's objective is recoverable; the quantity each policy reads is always identifiable, even where the paper does not name it as a signal; and the portability class follows from the mechanism in all sixteen cases. Eight of the sixteen require nothing of an arriving request beyond the request itself, five require a deadline or SLO class, two require a session identifier, and one requires a priority class.

One property does not fill. For no paper in the corpus could we determine the range the policy's own signal took during its evaluation. This is not a property we invented as a hurdle: it is the

difference between a policy that was tested and a policy that was merely present. A threshold never approached and a threshold approached but not crossed produce the same admission counts and the same downstream latencies, and no reported number distinguishes them.

The consequence for a reader is narrow and specific. The taxonomy will tell you which policies could apply to your deployment. It will not tell you whether the published evidence for any of them describes a policy that was actually exercised, because that evidence is not reported. Section 8.1 states the property as the third of seven reporting items, and Section 10 states the measurement it would take to establish which of those items are load-bearing.

3.8 The unclaimed intersection

Plotting the corpus (Figure 3) shows that agent-level × online × deadline-aware × per-tenant-fair is not fully occupied by any published policy. CONCUR is agent-level but offline and throughput-oriented. Chronos is deadline-aware but request-level. FairBatching is fair but task-level not tenant-level. SAGA [18] is the closest work: it schedules at the agent-workflow level with a bounded-deviation Agent Fair Share metric and reports 99.2% SLO attainment, but the paper does not establish that Agent Fair Share reduces to per-tenant fairness in multi-user settings, does not decompose the SLO into TTFT and TBT, and does not commit to an online-serving mode. We treat SAGA as a partial occupant and preserve OP1 as an open problem for the subset (agent + online + TTFT+TBT + per-tenant) that no published work satisfies simultaneously.

Re-deriving this after adding the portability axis leaves the cell empty, and surfaces a second gap beneath it. Among the sixteen admission-primary works, two are agent-level: CONCUR, which is Class A but offline and throughput-oriented, and Continuum, which is online but Class B and optimises job completion time rather than a deadline. So within the admission-primary set there is no agent-level policy that both serves online traffic and deploys without an engine fork. A reader running multi-turn agent workloads who cannot maintain a fork has no candidate at all, which is a sharper statement than the empty intersection and follows from a property no paper reports about itself.

We state this for the admission-primary set only. ThunderAgent [26] is agent-level and coded at ingress, which would occupy the cell, but its mechanism is program-level residency with eviction from high-bandwidth memory, and whether that is deployable outside the engine is not something we have established. It is the near-miss a reader should check first.

4 METHOD

We conducted a systematic review following an adapted PRISMA 2020 workflow. Figure 4 shows the identification, screening, eligibility, and inclusion counts.

Search dates and sources. Search dates ran from May 2022 (Orca) through September 2026. Sources: DBLP; arXiv (cs.DC, cs.LG, cs.OS, cs.PF categories); OpenReview forums for ICLR, NeurIPS, ICML, MLSys; USENIX conference proceedings for OSDI, NSDI, ATC, FAST; ACM Digital Library for SOSP, SoCC, ASPLOS, EuroSys, SIGMETRICS; IEEE Xplore for HPCA, ISCA; Google Scholar for cross-reference discovery.

Search strings. Admission-control terms (“admission control”, “queue management”, “load shedding”, “request rejection”, “request dropping”, “SLO-aware admission”, “throttling”, “backpressure”, “gating”) were combined with LLM-serving terms (“LLM inference”, “LLM serving”, “language model inference”, “agentic inference”, “continuous batching”, “KV cache”). Boolean expansion followed each hit’s related-work section for backward-citation discovery.

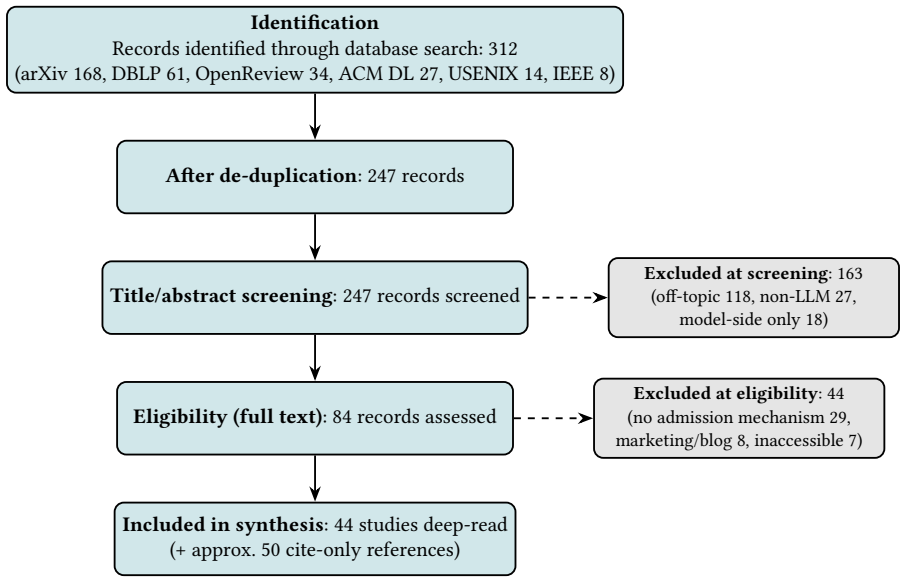


Fig. 4. PRISMA 2020 flow for the admission-control corpus. Counts as of 2026-09-11. Search protocol and per-paper protocol described in the subsections below.

Inclusion. A paper was included if all four conditions held: (i) published or preprinted between 2022-05-01 and 2026-09-11; (ii) primary application domain is LLM inference (not general DNN serving or generic queueing theory); (iii) either admission control is the primary contribution, or the paper is a top-venue scheduling / KV-management / fairness work whose mechanism has direct admission-adjacency; (iv) full text was accessible via arXiv HTML, venue PDF, or OpenReview.

Exclusion. A paper was excluded if any held: (i) pure model-side optimization (quantization, pruning, distillation) without a serving-time admission component; (ii) vision or speech inference without LLM applicability; (iii) marketing materials or blog posts without a peer-reviewed or arXiv-preprint anchor.

Tier structure. We organized the corpus into six tiers based on admission-primacy. Tier 1 (17 papers): admission-primary, one policy per paper. Tier 2 (10 papers): queueing theory or fairness-primary with admission-adjacency. Tier 3 (6 papers): KV-cache-management papers whose mechanism has admission implications. Tier 4 (9 papers): top-venue scheduling papers with admission as a secondary or missing component. Tier 5 (6 papers): foundational works cited for background. Tier 6 (5 papers): prior broad surveys used for differentiation. Total after inclusion and de-duplication: 44 papers deep-read; approximately 50 additional cite-only references.

Per-paper protocol. For each work we recorded venue, authors, arXiv or DOI identifier, code availability, taxonomy placement across all five axes, primary contribution, baseline set, evaluation setup (model, engine, workload, hardware), claimed headline numbers with configuration qualifiers, fidelity notes, and open questions surfaced. Notes were written in a common markdown template to make cross-work comparison tractable. The complete per-paper corpus is public at the survey's artifact repository.

Reviewer discussion evidence. For several open problems (Section 10) we cite public OpenReview discussion or venue meta-reviews as evidence that a gap is practitioner-recognized. Where a peer

reviewer explicitly asked a question that the paper's rebuttal did not answer, we treat that as grounded evidence for an open problem.

Non-experiments. We did not run experiments. Where papers report conflicting numbers, we report each source's number as stated with the operating-point qualifier the source specifies, and flag the conflict rather than adjudicate.

5 POLICY CLUSTERS

5.1 Request-level SLO-aware admission

QLM [46] is the canonical peer-reviewed admission-primary paper for online serving. It maintains virtual queues per model and formulates request scheduling as the linear program in Equation 5. A Request Waiting Time Estimator predicts per-request wait using Central Limit Theorem bounds over historical output-token distributions. Reported improvements are 40 to 90 percent SLO attainment lift and 20 percent (single-model) to 400 percent (multi-model) throughput gain over vLLM and EDF baselines on a heterogeneous 30 A10 plus 50 A100 cluster serving Mistral-7B, Vicuna-13B, and LLaMA-70B on ShareGPT. Code is public at https://github.com/QLM-project/QLM.

Chronos [37] derives a formal schedulability condition (Equation 6) for LLM inference. On a 7B model in a discrete-event simulator against 50,000 sampled Azure conversational-service requests, the reported separation is load-dependent. At 5× nominal load Chronos and the uncontrolled schedulers all reach 0.00 percent TTFT miss, and the difference is in tail latency and admission volume: Chronos admits 88.9 percent at a P99 TTFT of 161 ms, against 100 percent admitted at 400 ms for FCFS and EDF. At 10× load the uncontrolled schedulers collapse to 99.44 percent TTFT miss while Chronos holds 0.00 percent by admitting 53.9 percent. Rate limiting alone does not reproduce this: a throughput-matched token bucket (AC-FCFS) misses 15.40 percent at that same 53.9 percent admission, and 6.80 percent at the 5× operating point despite matching Chronos's admission volume there. The comparison is therefore a case for the formal test over rate matching rather than over scheduling as such. Simulator code is public at https://github.com/am-research/rtss-ttft-tbt.

SOLA [20] formalizes fine-grained scheduling as an LP over per-request and systemic state. Empirical scheduling raises SLO attainment from 45.5 percent to 99.4 percent and serves 1.04-1.27 × more requests. The constraint-relaxation mechanism at the α-percentile is an implicit admission decision hidden inside a scheduler.

FastServe [54] argues for token-granularity preemption rather than reject-on-arrival admission. The 31.4× average-latency throughput improvement is measured on OPT-175B across 16 A100 GPUs against vLLM v0.1.7 on ShareGPT. Code is public but runs on the SwiftTransformer engine.

SLOs-Serve [9] is the only paper in our corpus that handles per-stage-of-agent SLOs. Multi-SLO DP admits requests whose joint SLOs are attainable and declines the rest through soft admission. Reports 2.2× geometric-mean per-GPU serving capacity across six workload scenarios and five models.

ProServe [21] frames scheduling as unified multi-priority request scheduling. Admission emerges from the SlideBatching loop's budget-and-memory gates combined with a token-level deadline-aware gain function $f_{\text{TDG}}(r) = \sum_i w_r(i) \cdot \mathbb{I}[t_{r,i} < \tau_{\text{TTFT}} + (i-1)\tau_{\text{TPOT}}]$ that scores requests against their per-token deadlines. This is queue-boundary admission (QB) rather than pure ingress, and we classify it as such in Table 3. Baselines include vLLM-FCFS, Sarathi-FCFS, Sarathi-Priority, Weighted VTC, and FairBatching. No code released.

Scorpio [52] is the most recent admission-primary entry and splits the decision across three stages: admission, queue management, and batch selection. Its TTFT Guard reorders by least deadline first and rejects requests whose first-token deadline is unattainable, estimating attainability from the cumulative prefill time of everything already queued. Its TPOT Guard admits only when

the predicted per-token latency for the resulting virtual batch stays under the tightest TPOT target in the running set. Both guards read a predicted output length from a fine-tuned OPT-125M classifier over 100 bins, which places Scorpio in the dual-gate signal class with learned rather than analytic grounding. It is evaluated on vLLM with Llama-3.1-8B and Gemma-2-27B on four A100s, against vLLM, S3 and Mooncake, reporting up to 14.4 times goodput and 46.5 percent better SLO adherence under high load.

Two details matter beyond those numbers. First, Mooncake is itself an early-rejection admission mechanism, which makes Scorpio one of the few policy-versus-policy comparisons in the corpus (Table 5). Second, Scorpio computes goodput with every arriving request in the denominator, so a rejected request counts as an SLO miss. That is the discipline we adopt in Section 2.4 for goodput-under-admission, arrived at independently. A metric counting only admitted requests flatters shedding: refuse 95 percent of traffic, serve the rest perfectly, and it reads 100 percent. Two efforts converging on the same denominator by separate routes is the strongest argument available that it is the correct one.

5.2 Agent-session-level admission

CONCUR [8] identifies middle-phase thrashing in agentic batch inference. The controller applies the dual-gate condition of Equation 4. Headline gains are 4.09× throughput against SGLang on Qwen3-32B at 256 sessions, tensor parallelism 8, on 8 H100 GPUs, and 1.90× against SGLang on DeepSeek-V3 at 40 sessions, TP 16, 16 H100 GPUs. Against Continuum as a baseline, CONCUR shows 1.69× on Qwen3-32B (added in rebuttal). The meta-review states the paper targets offline throughput-oriented inference and that online serving is out of scope.

Continuum [32] manages KV retention rather than gating admission. Per-session time-to-live is set by maximizing expected benefit minus cost. Reports 1.12-3.66× JCT improvement across Llama-3.1, Gemma-3, and GLM-4.5 on SWE-Bench, BFCL, and OpenHands, with an internal-deployment peak of 8.18×. Public vLLM fork at https://github.com/Hanchenli/vllm-continuum.

ThunderAgent [26] treats disk, ports, and sandboxes as first-class scheduled resources alongside GPU. A central scheduler pauses and restores programs with explicit state migration. Integrated into NVIDIA Dynamo 2.0.

SAGA [18] schedules at the level of the entire agent workflow rather than per inference call, introducing Agent Execution Graphs that capture workflow structure across tool-call boundaries and predict KV cache reuse. The paper defines Agent Fair Share, a task-completion-time fairness metric with provable bounded-deviation guarantees, and reports 99.2% SLO attainment under multi-tenant interference. Admission is described as "session-affinity batching with work stealing" at the scheduler level; whether this reduces to ingress admission is not established, and per-tenant fairness (as opposed to per-task fairness) is not evaluated. We classify SAGA as agent-level, mid-flight, scalar-signal in Table 3. See Section 10 OP1 for the sharpened residual open problem.

5.3 Fairness-aware admission and scheduling

VTC [49] formalizes LLM-serving fairness through a cost function over tokens processed rather than requests, with default weights $w_p = 1$ for input and $w_q = 2$ for output. The theoretical guarantee bounds service difference between any two continuously-backlogged clients:

$$|W_f - W_g| \leq 2\max\left(w_p \cdot L_{\text{input}},\ w_q \cdot M\right) \tag{7}$$

where M is the max batch size, with a matching lower bound of $w_q \cdot M$ proving the 2× constant is tight. Empirically VTC halves max unfairness versus FCFS while matching throughput (779 versus 777 tok/s on 27 Chatbot Arena clients), whereas RPM(5) rate limiting collapses throughput to 340 tok/s. Artifact-evaluated code at https://github.com/Ying1123/VTC-artifact.

FairBatching [35] attacks fairness between prefill and decode tasks within a batch. Achieves 2.29× tail latency reduction, 20 percent single-node capacity improvement, and 54.3 percent cluster-level capacity improvement. Implementation is only 90 lines of invasive vLLM changes.

Equinox [34] argues that equal tokens do not imply equal user experience. Requires a Model of Prediction Ensemble to predict output length, latency, throughput, and GPU utilization per prompt. Thirteen percent fairness improvement measured against Equinox's own metric.

FairServe [27], from Microsoft on real Copilot production traces, combines Online Interaction Throttling (OIT) at ingress with Weighted Service Counting (WSC) for scheduling. OIT throttles at the LLM-interaction level when the KV cache is overloaded, and WSC selects requests from users who have received the least service, weighted by application priority. Reports 1.03–1.06× TTFT latency reduction. FairServe is the only work in our corpus that operates at the user/application-interaction level rather than at request, agent-session, or organizational tenant granularity; we classify it as tenant-level with a dagger (§6) and note that it does not decompose SLOs into TTFT+TBT deadlines. Source code not released as of paper submission.

5.4 Queueing-theoretic and algorithmic

Service-Induced Congestion [6] provides the theoretical justification (Section 2.5, Equation 3).

Flow-Controlled Scheduling [13] proves necessary and sufficient stability conditions using a static token-bucket. LIFO eviction inverts fairness for interactive workloads.

Fluid-WAIT [7] derives a threshold admission rule from a fluid-model equilibrium.

Shen et al. [48] prove $\tilde{O}(\sqrt{KT})$ regret for joint online configuration selection and admission control under an i.i.d. reward-resource assumption per configuration:

$$\mathrm{Reg}^{\mathrm{mix}}(T) \;\le\; C_1\alpha\sqrt{KT\cdot d\log T} + C_2\sqrt{T\log T} + KR_{\max} \tag{8}$$

The paper motivates via LLM serving but validates empirically on Alibaba cluster traces.

Jaillet et al. [23] target online scheduling under KV-cache constraints with monotonically growing per-request memory. Their memory-cognizant policies achieve competitive-ratio bounds under structured arrivals plus α-accurate predictions.

Mitzenmacher and Shahout [38] enumerate 25 open problems at the intersection of queueing theory and LLM serving.

Clairvoyant [51] is the sole entry in the serial-edge concurrency-context sub-branch. Predictive shortest-job-first using an XGBoost classifier over 19 lexical features delivers 70 to 76 percent short-request p_{50} latency reduction on an RTX 4090.

Ascendra [14] uses an SLO-risk predictor to promote requests between low-priority and high-priority queues.

Niyama [16] sits adjacent to admission rather than inside it. It co-schedules mixed workloads on shared infrastructure with fine-grained QoS classes and a hybrid fairness-efficiency priority policy, and under overload it applies selective request relegation, degrading service for lower classes rather than refusing work at the door. We include it because relegation and admission answer the same question with different verbs, and because its overload behaviour is directly comparable to that of a deferring admission policy.

5.5 KV-cache management adjacent

KVFlow [43] is pure cache management with steps-to-execution ranking over SGLang's radix tree. No admission logic. Complementary to CONCUR, not a substitute.

Cake [25] optimizes single-request TTFT via hybrid compute-versus-load KV decisions.

InferCept [1] chooses between preserving, swapping, and discarding KV during tool-call intercepts. Cost model prices GPU-memory-bytes-times-time locally; does not account for queue length, which Continuum identifies as its key limitation.

Pie [15] provides a programmable serving substrate with generation-loop handlers and KV page management. No cross-inferlet scheduler API and no admit-reject hook.

5.6 Scheduling papers with admission-adjacent components

We audited nine scheduling papers drawn from OSDI, SOSP, NSDI, ASPLOS, EuroSys, MLSys, and HPCA between 2022 and 2026. Table 2 summarizes what each does under overload. All nine locate the overload response inside the engine, redistributing work through batching, migration, placement, autoscaling or memory management. None takes a decision at the ingress, and none sets out to: these are scheduling and memory-management contributions, and refusing work at the door lies outside the design space each paper defines for itself. The observation is about where the research attention has gone, not about a shortcoming in these systems. Token Pools (CAIS 2026) sits outside this seven-venue set and does implement ingress admission at the API gateway; we discuss it separately below.

Table 2. Overload response mechanisms in nine LLM serving papers from the seven-venue set (OSDI, SOSP, NSDI, ASPLOS, EuroSys, MLSys, HPCA), 2022–2026. Each locates its overload response inside the engine; the ingress-admission column is empty across all nine, which reflects the scope each paper sets rather than an omission. Token Pools (CAIS 2026), outside this venue set, does gate at the API gateway and is discussed separately.

Paper	Venue	Overload mechanism	Ingress admission?
Orca	OSDI 2022	FCFS + KV-slot cap (implicit)	No
Sarathi-Serve	OSDI 2024	Chunked prefill batching	No
Llumnix	OSDI 2024	Live request migration	No
DistServe	OSDI 2024	P/D disaggregation, static split	No
Past-Future [17]	ASPLOS 2025	Memory projection at batch boundary	Internal only
Aegaeon [56]	SOSP 2025	Per-token autoscaling	No
Jenga [55]	SOSP 2025	Memory heterogeneity mgmt	No
AdaServe [33]	EuroSys 2026	SLO-customized spec decode	No
PASCAL [53]	HPCA 2026	Runtime phase detection	No

Past-Future's memory-projection gate is the closest of the nine to admission, and it operates on already-queued requests rather than at ingress. Taken with Token Pools, which sits outside the seven-venue set and does gate at the gateway, the pattern suggests that ingress admission has been treated as an infrastructure concern rather than an engine-level research question. That is the gap this survey addresses: not that these systems are missing something they should have, but that the decision they leave to the deployment has received little systematic study.

Token Pools (CAIS 2026): partial counter-example. Cunningham [12] introduces token pools, a control-plane abstraction expressing capacity as inference-native entitlements (token throughput, KV cache, concurrency). Admission is enforced at the API boundary: "The auth service intercepts every request before it reaches the backend. On each request, it validates the API key against an entitlement… and makes an admission decision based on multiple factors." Requests whose priority falls below the admission threshold "are rejected with HTTP 429 and a `Retry-After` header, allowing clients to back off gracefully." The same entitlements drive autoscaling decisions through a unified capacity model, directly addressing our Open Problem 5 (coupling admission and

autoscaling). The evaluation is single-node (DGX Spark, Qwen3-8B) with no released code, so we rate the evidence M. Token Pools tightens but does not close the top-venue-scheduling gap: it sits outside the seven-venue OSDI/SOSP/NSDI/ASPLOS/EuroSys/MLSys/HPCA set, and its evaluation scale is orders of magnitude below what those venues typically require.

6 MASTER CLASSIFICATION

Table 3 classifies twenty-eight of the fifty-five works in the corpus along the three orthogonal taxonomy levels (Unit, Moment, Signal grounding), plus a GRADE-adapted evidence rating for the strength of empirical support behind each work's headline claims. Unit codes: REQ request, AGT agent-session, TEN tenant. Moment codes: ING ingress, QB queue boundary, MF mid-flight (during decode). Signal codes: SCA scalar, DG dual-gate, LP linear-program, WCRT formal worst-case-response-time bound.

What the table does not classify. Twenty-seven corpus works carry no row. Twelve are foundational systems, textbooks, or prior surveys, for which an admission taxonomy is not defined. Nine are scheduling papers from the seven-venue slice that contain no admission mechanism to classify, which is the nine-of-nine finding of Section 5 rather than an omission here. The remaining six are AQUA [28], Chiron, Mooncake, Niyama [16], Throughput-Optimal Scheduling and Beyond Prediction, each of which touches admission indirectly: AQUA and Mooncake through reclamation and early rejection, Chiron through autoscaling, Niyama through relegating lower QoS classes rather than refusing them, and the last two through queueing-theoretic results that do not specify a decision rule. Mooncake warrants a specific note, since it states a "prediction-based early rejection policy" and appears as a baseline in Table 5. We leave it unclassified on the narrower ground that admission is a component of a disaggregation architecture rather than the paper's contribution: the rejection policy is described but not evaluated as a policy, with no ablation isolating its effect and no comparison against an alternative admission rule. It is therefore an admission mechanism we can name but not place on the Signal axis with the same evidence we have for the entries in Table 3.

GRADE rubric (operationalized). We assign each work one of four evidence levels using four criteria: (1) evaluation medium (real hardware vs simulator vs theoretical); (2) number of independent baselines evaluated; (3) code availability (public + artifact-evaluated vs public unevaluated vs closed); (4) workload realism (production trace or public benchmark vs synthetic). A work is assigned:

H (high) real hardware, three or more independent baselines including at least one from a different venue, artifact-evaluated public code, and evaluation on a public benchmark or production trace.

M (moderate) real hardware, at least one independent baseline, public code (not necessarily artifact-evaluated), and either a public benchmark or a well-documented synthetic workload.

L (low) simulator-only, or unreleased code, or single-baseline against an unnamed reference, or synthetic workload without documented parameters.

VL (very low) position paper, tutorial, or theoretical result without empirical evaluation.

The rubric is stricter than reviewer-perceived at borderline cells. QLM earns H because its artifact is Artifact-Evaluated, baselines include vLLM+EDF and SHEPHERD, and evaluation runs on Chatbot-Arena traces on a heterogeneous 80-GPU cluster. Chronos earns M rather than H because the evaluation is simulator-only despite artifact release. VTC earns H because the OSDI artifact evaluation and clean cross-baseline comparison satisfy all four criteria. Per-paper justifications are included in the artifact repository's per-paper coding sheet.

Table 3. Master classification of admission-primary and admission-adjacent works. See legend above the table for codes. "–" means the axis does not apply.

System / Paper	Venue	Year	Unit	Moment	Signal	GRADE
Request-level SLO-aware admission (§5)						
QLM	SoCC	2024	req	ing	lp	H
Chronos	Frontiers CS	2026	req	ing	wcrt	M
SOLA	MLSys	2025	req	mf	lp	M
FastServe	NSDI	2026	req	mf	sca	M
SLOs-Serve	preprint	2025	req	ing	lp	L
Fluid-WAIT	preprint	2025	req	ing	sca	L
Scorpio	WISE	2026	req	ing	dg	M
Ascendra	preprint	2025	req	qb	sca	L
Flow-Ctrl scheduling	preprint	2026	req	ing	wcrt	L
Service-Induced Cong.	preprint	2026	req	ing	wcrt	M
Jaillet et al.	preprint	2026	req	ing	lp	M
Shen et al.	ICML	2026	req	ing	lp	M
ProServe [22]	preprint	2025	req	qb	lp	L
Agent-session-level admission						
CONCUR	ICML	2026	agt	ing	dg	M
Continuum	ICLR WS	2026	agt	mf	sca	M
ThunderAgent	ICML	2026	agt	ing	sca	L
SAGA	preprint	2026	agt	mf	sca	L
Fairness-oriented						
VTC	OSDI	2024	ten	mf	sca	H
FairBatching	preprint	2025	req	mf	sca	L
Equinox	preprint	2025	ten	ing	sca	L
FairServe	preprint	2024	ten†	ing	dg	M
Gateway-level admission + autoscaling						
Token Pools	CAIS	2026	ten	ing	lp	M
Serial-edge concurrency context						
Clairvoyant	preprint	2026	req	ing	sca	L
KV-management adjacent (no ingress admission)						
KVFlow	NeurIPS	2025	–	mf	sca	M
Cake	ICML	2025	–	mf	sca	M
InferCept	ICML	2024	–	mf	sca	M
Pie	SOSP	2025	–	mf	–	L
Position / tutorial						
Mitzenmacher & Shahout	SIGMETRICS	2025	–	–	–	VL

The nine scheduling papers from the OSDI/SOSP/NSDI/ASPLOS/EuroSys/ MLSys/HPCA seven-venue slice (Orca, Sarathi-Serve, Llumnix, DistServe, Past-Future, Aegaeon, Jenga, AdaServe, PASCAL) are omitted from Table 3 because none includes an ingress-level admission decision at all; they are summarized separately in Table 2. TokenScale [30] is also omitted despite its LLM-serving scope: careful reading confirms it is a pure autoscaling framework (Token Velocity signal drives prefiller/decoder scaling only) with no admission-control component. Chiron [45] is a hierarchical autoscaler with no admission component and appears only under Open Problem 5.

Dagger note on FairServe. FairServe [27] operates at the user- and application-interaction level rather than at any of our three canonical unit codes (REQ, AGT, TEN). We mark it TEN† because a Microsoft-CoPilot "application" is closer to a tenant than to an individual request or agent session, but the classification is genuinely lossy for this work; readers checking the four-property intersection should note that FairServe satisfies the online axis and a weighted per-user fairness axis, but does not decompose SLOs into TTFT and TBT deadlines and does not operate at the agent-session level. Placement was chosen after considering the reviewer's explicit question 3 on the four-property intersection.

7 ADMISSION-SIGNAL OVERHEAD SYNTHESIS

The corpus supports one cross-paper empirical synthesis that no individual paper reports: the tradeoff between the *cost* of computing an admission signal at ingress and the *goodput lift* that signal delivers over a first-come-first-serve baseline. We call this the *admission-signal overhead*. Five anchor points from the corpus are shown in Table 4, each carrying its operating point verbatim from the source paper so that the reader can see the heterogeneity of contexts across which we are reasoning.

Table 4. Admission-signal overhead: cost of computing an admission decision (per request, best-case) versus lift reported in the source paper over that paper's own FCFS baseline. Every row reports the exact operating context from its source; **no normalization is applied across rows** and cross-row comparison is not justified without head-to-head evaluation.

Anchor (signal class)	Model / engine	Hardware	SLO tuple / workload	Cost	Lift over FCFS
Fluid-WAIT (SIG-SCA)	Llama-2-7B / vLLM	1×A100 40	TTFT 500 ms / synthetic Poisson	tens of µs	10–20% goodput
Clairvoyant (SIG-SCA)	Llama-3.1-8B / Ollama	RTX 4090	p50 latency / ShareGPT	tens of µs	70–76% short-request p50 latency reduction
CONCUR (SIG-DG)	Qwen3-32B / SGLang	8×H100	JCT / BrowserComp, 256 sessions	<100 µs + AIMD update	4.09× throughput vs SGLang
QLM (SIG-LP)	Mistral-7B + Vicuna-13B + LLaMA-70B / vLLM	30×A10 + 50×A100	TTFT tail / ShareGPT	≈5 s Gurobi solve at 400 K queued	40–90% SLO-attainment vs vLLM; 20% throughput single-model, 50–400% multi-model, vs vLLM+EDF
Chronos (SIG-WCRT)	7B ref. model / discrete-event sim	simulator	TTFT deadline / 50 K Azure requests	µs (closed-form)	TTFT miss 99.44%→0.00% at 5× load (sim)

What Table 4 does and does not support. The five anchors span different models, engines, hardware, SLO definitions, and workloads. Reading down the "Lift" column suggests that costlier signals return larger lifts, but this pattern is *directional at best* because every anchor is a different-workload single-baseline comparison to that anchor's own FCFS. We explicitly do not claim: (a) a functional form of cost versus lift; (b) a crossover point beyond which additional signal complexity ceases to pay off; (c) that any pair of anchors would retain its ordering under head-to-head evaluation. The synthesis is provided so the reader can see the shape of the tradeoff and the absence of normalized evidence for it.

GRADE certainty. We rate this synthesis LOW using the rubric in Section 6. Each anchor cites its source paper's own numbers under that paper's own operating point; no head-to-head comparison exists in the literature (Table 5). Closing this identification gap is what the reporting items in Section 8 are meant to enable, and establishing which of them are load-bearing is Open Problem 7 (Section 10).

8 REPRODUCIBILITY AND COMPARISON METHODOLOGY

We audited public code availability across our corpus, scoped to the tiers where releasing an implementation is a well-formed expectation. Seven of the sixteen admission-primary papers have released code as of September 2026 (QLM, Chronos, FastServe, Shen et al., Continuum, Clairvoyant, AQUA), as have five of the six KV-cache-adjacent works (KVFlow, InferCept, Pie, ThunderAgent, Cake) and one of the ten fairness and queueing-theory works (VTC). Two qualifications apply to the admission-primary count: Chronos released a discrete-event simulator rather than a real-engine implementation, and AQUA is included in this corpus as the canonical preemption-based foil to admission control rather than as an admission policy itself.

These rates are higher than the "reproducibility crisis" framing common in prior surveys would suggest, and the adjacent literature releases code more often than the admission literature does. We report rates per tier rather than a single corpus-wide figure because the full corpus includes foundational works, textbooks and prior surveys, for which code release is not a meaningful expectation; a denominator spanning them would not be interpretable.

The more consequential gap is at the methodology level. Table 5 shows the baseline set used by each admission-primary paper. No two papers share a baseline *set*; the baselines that do recur across papers are unconfigured serving engines and classical scheduling disciplines, not admission policies.

Table 5. Baseline overlap across admission-primary papers. The most-used baseline (vLLM) appears in six papers and Sarathi in four; no other appears in more than two; and no two papers share the same baseline set. Read by what the columns contain: vLLM, SGLang, Sarathi, DistServe and Orca are serving engines, so a tick there measures a policy against the absence of admission control rather than against an alternative to it, and EDF, FCFS and SJF are classical disciplines rather than published systems. SOLA's Sarathi entry is marked *re-impl.* because it reimplements chunked prefill on vLLM rather than comparing against upstream Sarathi-Serve. Among published admission systems, SHEPHERD [58] is used as a baseline by two papers (QLM and Ascendra); AC-FCFS also appears twice, but it is a throughput-matched token bucket constructed as an ablation rather than a system published in its own right. Scorpio's comparison against Mooncake is the corpus's clearest instance of one admission mechanism measured against another. Rows for CONCUR, Continuum, SAGA, and Token Pools are also omitted from column headers (a paper cannot baseline against itself); Continuum appears only as CONCUR's rebuttal-added comparator. Empty rows for SAGA and Token Pools indicate baselines that are not extractable from the current text of those preprints.

Paper	vLLM	SGLang	Sarathi	DistServe	Orca	EDF	FCFS	SJF	SHEPHERD	Weighted VTC	Continuum	AC-FCFS
QLM (SoCC 2024)	✓				(cite)	✓	(impl.)		✓			
CONCUR (ICML 2026)		✓									✓	✓
Chronos (Frontiers 2026)						✓	✓					✓
SOLA (MLSys 2025)	✓		re-impl.					✓				
FastServe (NSDI 2026)	✓											
SLOs-Serve (2025)	✓		✓	✓								
FairBatching (2025)			✓									
Ascendra (2025)									✓			
ProServe (2025)	✓		✓				✓			✓		
SAGA (2026)	(baselines not extractable from preprint abstract; body-text extraction failed via WebFetch)											
Token Pools (CAIS 2026)	(single-node evaluation; no external policy baselines reported)											
Fluid-WAIT (2025)	(unnamed "widely used baselines")											
Scorpio (WISE 2026)	✓	+ S3 and Mooncake, the latter itself an early-rejection mechanism										

Reading the literature top to bottom, a practitioner cannot answer "which admission policy wins on my workload."

8.1 Seven reporting items that vary across the corpus

Reading the corpus for what would be needed to place a result against a deployment, seven properties recur as the points of divergence. Six are reported inconsistently. The seventh, the observed

range of the signal the policy reads, we did not find reported by any of the sixteen admission-primary papers.

That asymmetry is worth stating plainly, because it is not what we expected to find. Placing all sixteen papers on the axes of Section 3 succeeds on every axis but one: unit, setting, SLO-awareness, the quantity the policy watches, the metadata a request must carry, and whether the policy is deployable in front of an unmodified engine are all recoverable from the published text. Ten of the sixteen are deployable without modifying the engine; eight require nothing of a request beyond the request itself. Selection fails on a single missing fact, and it is the fact that determines whether a reported improvement is interpretable at all.

All seven concern disclosure of what was done rather than prescription of what to run, so none of them asks a paper to change its evaluation.

(1) **Capacity-relative load**. Offered load expressed as a multiple of the deployment's measured serveable capacity, rather than as a raw arrival rate or an unquantified "high load". Two papers reporting the same arrival rate on different hardware may be operating in different regimes.
(2) **Calibrated deadlines**. SLO targets expressed relative to unloaded latency on the same model and hardware. The difficulty of meeting a fixed millisecond target varies with the deployment, so absolute targets are not comparable across papers.
(3) **Signal liveness**. The observed range of the quantity the policy reads, over the evaluation. Not one of the sixteen admission-primary papers reports it.[1] Without it, a policy whose signal never approached its threshold cannot be distinguished from one that evaluated a varying signal and declined to reject (Section 3.6).
(4) **Repeats and spread**. The number of repeats and the observed spread, rather than a single run or a median alone.
(5) **Parameter provenance**. For each parameter, whether it was fitted on the evaluated deployment, inherited from the source paper, or left at a default. Policies carrying hardware-dependent cost constants cannot otherwise be reproduced.
(6) **Metric definition**, in particular the goodput denominator: whether rejected requests count as misses (Definition 2) or are excluded. The two conventions yield different numbers for the same run, and the difference grows with the rejection rate.
(7) **Configuration disclosure**. Model, engine version, hardware, workload subset, offered load, SLO tuple, and the identity of the baseline actually run.

We do not claim this set is minimal or sufficient, or that its items matter equally; it records where the corpus diverges, stated so that a reader can check a paper against it. Determining which of the seven are load-bearing, in the sense that omitting one allows a reported result to move without a reader detecting the change, is not answerable by reading. It requires running the same policy under controlled variation of each item in turn, and we state it as Open Problem 7.

Per-paper coding sheets, taxonomy placements, and inclusion and exclusion decisions for this survey are released publicly, to support third-party audit and to reproduce the master table (repository withheld for anonymous review; the coding sheets are included as supplementary material).

[1] This is an absence across the evaluation sections we extracted, not a guarantee that no paper mentions a range anywhere. We recorded each paper's evaluation setup, including the quantity its policy reads; none stated the range that quantity took. A figure axis showing, say, cache occupancy over time would arguably satisfy the item and would not have been captured by this procedure.

9 PRODUCTION EVIDENCE

Real inference-fleet incidents map directly onto admission failure modes. Table 6 lists eight public postmortems and identifies the leading signal that would have prevented each. Each row carries a source URL and a confidence annotation: V (verified) means the failure mode is explicitly named in the linked postmortem; I (inferred) means the failure mode is our reading of an official postmortem whose wording is looser; H (hypothesized) means the mapping to an admission-signal gap is our own analytic reconstruction, not stated by the operator.

Table 6. Production incidents mapped to admission signals. Confidence: V verified from postmortem; I inferred from postmortem; H hypothesized (our reconstruction). The Azure 2016 entry is marked H because Azure's public status history no longer retains root-cause analyses from that period, so the account cannot be checked against a primary source.

Incident	Failure mode	Trusted signal (was)	Leading signal (needed)	Source	Conf.
Azure Mar 2016	Control-plane inferred backend unavailability, returned 429 for 3.5–4 h	HTTP 400 (spurious)	Cross-validated backend liveness	archive not retained	H
OpenAI DALL·E queue collapse	Work admitted past client cancellation timeout	Queue depth	Queue age at admission	[39]	I
OpenAI Nov 8 2023	Routing nodes crossed soak point; adding replicas insufficient	Utilization	Rate of change of utilization	[40]	V
OpenAI Dec 11 2024	Telemetry API storm; stale DNS for 20 min then failed at TTL expiry	Aggregate utilization dashboards	Control-plane health independent of data-plane cache	[41]	V
OpenAI Apr 2025	Control-plane OOM; failover overloaded second cluster; misleading errors	Cluster availability	Correct 429/503 propagation on failover	[42]	V
Anthropic Jan 21 2026	Insufficient compute for demand; latency then errors for ~90 min	Metrics dashboard refresh cycle	Sub-minute capacity-constraint detection	[4]	I
Anthropic Aug 29 2025	LB moved short-context traffic to servers configured for 1M-token contexts	Pool-level utilization	Per-configuration utilization decomposition	[5]	V
Anthropic HTTP 529 semantics	Provider-overloaded distinct from client-rate-limited	Single 429/503 signal	Dedicated 529 semantics	[3]	V

9.1 Practitioner checklist

For operators evaluating an admission policy: (i) what does the policy admit against, and can that signal be corrupted by the failure modes in Table 6? (ii) does the policy return 429, 503, or 529, and does the choice match your client retry logic? (iii) how quickly does the policy respond to a capacity change smaller than your metric-dashboard refresh interval? (iv) does the policy see per-configuration utilization, or only pool-level averages?

10 OPEN PROBLEMS

Seven high-signal open problems. Each cites concrete evidence from either published peer review or the primary source's own limitations.

OP1: Agent-level admission with per-tenant fairness under online arrivals. The CONCUR reviewer discussion at ICML 2026 (OpenReview `mmdqUrEY24`) contains this exchange: Reviewer Afcg maintained a weak-accept score with the final justification "the underlying problem can be addressed through more effective request-level scheduling." CONCUR added a Continuum comparison in rebuttal but did not add a request-level baseline. SAGA [18] partially closes the gap by scheduling at agent workflow granularity with a bounded-deviation Agent Fair Share metric, but the paper does not evaluate per-tenant fairness across concurrent users, does not decompose SLOs into TTFT and

TBT, and does not commit to an online serving mode. The sharpened open problem is: an agent-level policy that admits under online arrivals, respects both TTFT and TBT deadlines per stage, and provides proven per-tenant (not per-task) fairness bounds.

OP2: Online serving admission for agentic workloads. The CONCUR meta-review states the paper targets offline throughput-oriented agentic inference. Continuum and ThunderAgent also target offline batch. SAGA reports online SLO attainment but does not evaluate under bursty arrivals with reject/defer accounting. An admission policy for agentic workloads that maintains stable goodput under Poisson-plus-burst arrivals and reports reject/defer/admit rates with per-agent-class breakdown remains open.

OP3: Multi-tenant admission with per-tenant SLOs. VTC handles fairness in served tokens. FairBatching handles fairness between prefill and decode. Neither handles per-tenant deadline SLOs with tenant-aware admission.

OP4: Reasoning-trace-aware admission at ingress. PASCAL addresses phase-aware scheduling for reasoning-augmented LLMs, but its phase detection is entirely runtime. Ingress-level reasoning-versus-chat classification is unclaimed.

OP5: Cross-validation of coupled admission and autoscaling. Chiron autoscales replicas; QLM (same research group) does per-replica SLO-aware queueing. Token Pools [12] directly couples entitlement-based admission with autoscaling, but the evaluation is single-node (DGX Spark, Qwen3-8B) with no released code, so the interaction regime is unmeasured at cluster scale. The residual open problem becomes: cross-validate a Token-Pools-style coupled admission + autoscaling policy against QLM + Chiron on the same multi-model multi-tenant cluster trace.

OP6: Formal schedulability with LLM-specific WCRT.. Chronos derives closed-form WCRT bounds under a linear kernel model with roofline-derived parameters, never measured on real silicon. Probabilistic WCET analysis and mixed-criticality models are open.

OP7: Which reporting items are load-bearing? Seven of the sixteen admission-primary papers have released code, which weakens the "no artifacts" framing. The remaining obstacle to comparison is the absence of a shared account of what must be reported for two results to be read against each other. Section 8.1 identifies seven items that diverge across the corpus. Their identification does not establish that they matter.

The open problem is to determine, for each item, whether omitting it permits a reported result to move without a reader detecting the change. Answering it requires running a fixed policy on a fixed deployment under controlled variation of each item in turn, and observing whether the headline number moves while the paper's disclosure stays constant. An item for which no such variation can be constructed would not be load-bearing, and the set should shrink accordingly.

Item 3 (signal liveness) is the clearest case to test, since we found no paper reporting it: the question is not whether papers disagree but whether the omission changes what a reader should conclude. Companion work by the present authors, in preparation, takes this as its subject; it is not cited here so as to preserve anonymity.

11 CONCLUSION

Admission control for LLM inference serving is a distinct sub-problem that governs whether downstream batching, KV management, and preemption have the capacity headroom to succeed. Between 2022 and 2026 the field produced thirteen admission-primary works at peer-reviewed top venues plus one accepted workshop paper and more than a dozen preprints. We surveyed fifty-five works across admission-primary, admission-adjacent, and scheduling clusters.

Our five-axis taxonomy with five sub-branches organizes the corpus without redundancy. The intersection agent-level × online × (TTFT+TBT-deadline-decomposed) × per-tenant-fair remains unoccupied; SAGA is a partial occupant on the agent-level and fairness axes only (Figure 3).

Nine scheduling papers from the seven-venue slice (OSDI, SOSP, NSDI, ASPLOS, EuroSys, MLSys, HPCA) place their overload response inside the engine rather than at the ingress (Table 2); Token Pools (CAIS 2026) sits outside this slice and does gate at the gateway. Production postmortems (Table 6) show that overload originating from a mis-specified admission signal propagates faster than downstream batching, migration, or autoscaling can absorb, so these techniques are insufficient on their own.

Reproducibility of individual artifacts is higher than commonly framed, at seven of the sixteen admission-primary papers. Reproducibility of cross-paper comparison is essentially zero (Table 5). The seven reporting items in Section 8.1 are what would enable closing it.

Seven open problems each carry either public peer-review evidence or the primary source's own limitation statement. The most tractable candidates for near-term progress are OP1 (agent-level admission with per-tenant fairness under online arrivals, sharpened by SAGA as partial occupant), OP2 (online admission for agentic workloads with reject/defer accounting), OP5 (cross-validation of coupled admission and autoscaling on top of Token Pools), and OP7 (which reporting items are load-bearing).

REFERENCES

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *International Conference on Machine Learning (ICML 2024)*. arXiv:2402.01869 https://github.com/WukLab/InferCept

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*.

[3] Anthropic. 2026. Anthropic API Documentation: Errors. https://docs.anthropic.com/en/api/errors.

[4] Anthropic. 2026. Anthropic incident report: January 21 2026 capacity miss. https://status.claude.com/incidents/yrxt885v63jw.

[5] Anthropic Engineering. 2025. A postmortem of three recent issues. https://www.anthropic.com/engineering/a-postmortem-of-three-recent-issues.

[6] Ruicheng Ao, Jing Dong, Gan Luo, and David Simchi-Levi. 2026. Service-Induced Congestion in Memory-Constrained LLM Serving. *arXiv preprint arXiv:2606.15555* (2026).

[7] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Chao Wang. 2025. Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints. *arXiv preprint arXiv:2504.11320* (2025).

[8] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. 2026. CONCUR: High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control. In *International Conference on Machine Learning (ICML 2026)*. arXiv:2601.22705 https://openreview.net/forum?id=mmdqUrEY24

[9] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B. Gibbons. 2025. SLOs-Serve: Optimized Serving of Multi-SLO LLMs. *arXiv preprint arXiv:2504.08784* (2025).

[10] Cloudflare. 2017. How we built rate limiting capable of scaling to millions of domains. https://blog.cloudflare.com/counting-things-a-lot-of-different-things/.

[11] Cloudflare. 2022. Introducing Advanced Rate Limiting. https://blog.cloudflare.com/advanced-rate-limiting/.

[12] William J. Cunningham. 2026. Token Management in Multi-Tenant AI Inference Platforms. In *ACM Conference on AI and Agentic Systems (CAIS '26)*. ACM. https://arxiv.org/abs/2603.00356 System name: Token Pools. arXiv:2603.00356.

[13] Zhuolun Dong and Junyu Cao. 2026. Flow-Controlled Scheduling for LLM Inference with Provable Stability Guarantees. *arXiv preprint arXiv:2604.11001* (2026).

[14] Sameh Elnikety et al. 2025. Ascendra: Dynamic Request Prioritization for Efficient LLM Serving. *arXiv preprint arXiv:2504.20828* (2025).

[15] In Gim et al. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '25)*. arXiv:2510.24051 https://github.com/pie-project/pie

[16] Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, and Ramachandran Ramjee. 2025. Niyama: Breaking the Silos of LLM Inference Serving. arXiv:2503.22562 [cs.LG] https://arxiv.org/abs/2503.22562

[17] Ruihao Gong et al. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25), Volume 2*. 798–813. https://doi.org/10.1145/3676641.3716011

[18] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. *arXiv preprint arXiv:2605.00528* (2026). https://arxiv.org/abs/2605.00528 v2 revised 2026-06-19.

[19] Morgan Heisler, Zahra Yousefijamarani, Xinglu Wang, et al. 2025. *LLM Inference Scheduling: A Survey of Techniques, Frameworks, and Trade-offs*. Technical Report. TechRxiv.

[20] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Xuefei Ning, Guohao Dai, Shengen Yan, Yun Liang, and Yu Wang. 2025. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Proceedings of Machine Learning and Systems (MLSys 2025)*. https://openreview.net/forum?id=ubIvpetAd6

[21] Weizhe Huang, Tao Peng, Tongxuan Liu, Donghe Jin, Kang Meng, Xianzhe Dong, and Ke Zhang. 2025. ProServe: Unified Multi-Priority Request Scheduling for LLM Serving. *arXiv preprint arXiv:2512.12928* (2025). https://arxiv.org/abs/2512.12928 v2 revised 2026-06-12.

[22] Weizhe Huang, Tao Peng, Tongxuan Liu, Donghe Jin, Kang Meng, Xianzhe Dong, and Ke Zhang. 2026. ProServe: Unified Multi-Priority Request Scheduling for LLM Serving. *arXiv preprint arXiv:2512.12928* (2026).

[23] Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. 2026. Online Scheduling for LLM Inference with KV Cache Constraints. *arXiv preprint arXiv:2502.07115* (2026).

[24] Rajendra K. Jain, Dah-Ming Chiu, and William R. Hawe. 1984. A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. *DEC Research Report TR-301* (1984).

[25] Jiayi Jin et al. 2025. Compute Or Load KV Cache? Why Not Both?. In *International Conference on Machine Learning (ICML 2025)*. arXiv:2410.03065 System name: Cake.

[26] Woosung Kang et al. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. In *International Conference on Machine Learning (ICML 2026)*. arXiv:2602.13692

[27] Redwan Ibne Seraj Khan, Kunal Jain, Haiying Shen, Ankur Mallick, Anjaly Parayil, Anoop Kulkarni, Steve Kofsky, Pankhuri Choudhary, Renée St. Amant, Rujia Wang, Yue Cheng, Ali R. Butt, Victor Rühle, Chetan Bansal, and Saravan Rajmohan. 2024. Ensuring Fair LLM Serving Amid Diverse Applications. *arXiv preprint arXiv:2411.15997* (2024). https://arxiv.org/abs/2411.15997 System name: FairServe.

[28] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. 2025. AQUA: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains. In *Proceedings of the 30th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2025)*. https://doi.org/10.1145/3676641.3715983 arXiv:2407.21255

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*.

[30] Ruiqi Lai, Hongrui Liu, Chengzhi Lu, Zonghao Liu, Siyu Cao, Siyang Shao, Yixin Zhang, Luo Mai, and Dmitrii Ustiugov. 2025. TokenScale: Timely and Accurate Autoscaling for Disaggregated LLM Serving with Token Velocity. *arXiv preprint arXiv:2512.03416* (2025). https://arxiv.org/abs/2512.03416

[31] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. LLM Inference Serving: Survey of Recent Advances and Opportunities. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–8. arXiv:2407.12391

[32] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2026. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv preprint arXiv:2511.02230* (2026). https://github.com/Hanchenli/vllm-continuum Accepted as poster at ICLR 2026 Workshop on Lifelong Agents.

[33] Xin Liao et al. 2026. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. https://doi.org/10.1145/3767295.3769315

[34] Ying Lu et al. 2025. Equinox: Holistic Fair Scheduling in Serving Large Language Models. *arXiv preprint arXiv:2508.16646* (2025).

[35] Hongtao Lyu, Boyue Liu, Mingyu Wu, and Haibo Chen. 2025. FairBatching: Fairness-Aware Batch Formation for LLM Inference. *arXiv preprint arXiv:2510.14392* (2025).

[36] Hanbo Ma, Zhongguo Li, Junan Wang, Chunyi Cao, Donglin Li, Jun Yang, and Zhengtao Ding. 2026. A Control-Oriented Survey of Load Balancing for LLM Inference Serving. SSRN preprint. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6516666

[37] Amine Marref, Khaled Tarmissi, and Hassene Chaibi. 2026. Formal Schedulability Analysis for LLM Inference: TTFT and TBT Deadline Guarantees via Response-Time Theory. *Frontiers in Computer Science* 8 (2026). https://doi.org/10.

3389/fcomp.2026.1873627
[38] Michael Mitzenmacher and Rana Shahout. 2025. Queueing, Predictions, and LLMs: Challenges and Open Problems. *SIGMETRICS 2025 Tutorial (arXiv:2503.07545)* (2025).
[39] OpenAI. 2023. OpenAI incident report: DALL·E queue collapse. https://status.openai.com/incidents/rllgb3r2rqsf.
[40] OpenAI. 2023. OpenAI incident report: November 8 2023 tipping point. https://status.openai.com/incidents/00fpy0yxrx1q.
[41] OpenAI. 2024. OpenAI incident report: December 11 2024 delayed cache cliff. https://status.openai.com/incidents/ctrsv3lwd797.
[42] OpenAI. 2025. OpenAI incident write-up: April 2025 failover. https://status.openai.com/incidents/01JMYB59DYSH9TF6JV52TWBGRG/write-up.
[43] Zaifeng Pan et al. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. In *Advances in Neural Information Processing Systems (NeurIPS 2025)*. arXiv:2507.07400 https://github.com/PanZaifeng/KVFlow
[44] Pratyush Patel et al. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE International Symposium on Computer Architecture (ISCA '24)*. 118–132.
[45] Archit Patke, Dhemath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron. *arXiv preprint arXiv:2501.08090* (2025). https://arxiv.org/abs/2501.08090
[46] Archit Patke, Dhemath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. Queue Management for SLO-Oriented Large Language Model Serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*. 18–35. https://doi.org/10.1145/3698038.3698523 arXiv:2407.00047
[47] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST '25)*.
[48] Owen Shen, Haoran Xu, Yinyu Ye, Peter Glynn, and Patrick Jaillet. 2026. A Two-Layer Framework for Joint Online Configuration Selection and Admission Control. In *International Conference on Machine Learning (ICML 2026)*. arXiv:2602.07663
[49] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. arXiv:2401.00588 https://github.com/Ying1123/VTC-artifact
[50] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*.
[51] Aravind Sundaresan. 2026. Clairvoyant: Predictive Shortest-Job-First Admission for Serial LLM Inference. *arXiv preprint arXiv:2606.07248* (2026). https://github.com/Aravind0403/clairvoyant-scheduler
[52] Yinghao Tang, Tingfeng Lan, Bo Pan, Xiuqi Huang, Hui Lu, and Wei Chen. 2026. Scorpio: Serving Right Requests at the Right Time for Heterogeneous SLOs in LLM Inference. In *27th International Conference on Web Information Systems Engineering (WISE 2026)*. https://arxiv.org/abs/2505.23022 arXiv:2505.23022; accepted version.
[53] Yubo Wang et al. 2026. PASCAL: A Phase-Aware Scheduling Algorithm for Serving Reasoning-based Large Language Models. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. arXiv:2602.11530
[54] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. Fast Distributed Inference Serving for Large Language Models. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. arXiv:2305.05920 https://github.com/LLMServe/FastServe
[55] He Wu et al. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. In *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '25)*. https://doi.org/10.1145/3731569.3764823 arXiv:2503.18292
[56] Ennan Yang et al. 2025. Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market. In *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '25)*. https://doi.org/10.1145/3731569.3764815
[57] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association. https://www.usenix.org/conference/osdi22/presentation/yu
[58] Hong Zhang et al. 2023. SHEPHERD: Serving DNNs in the Wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. 787–808.
[59] Yingjun Zhen et al. 2025. Taming the Titans: A Survey of Efficient LLM Inference Serving. In *Proceedings of the 18th International Natural Language Generation Conference (INLG 2025)*. arXiv:2504.19720
[60] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *18th USENIX*

Symposium on Operating Systems Design and Implementation (OSDI '24). arXiv:2401.09670